

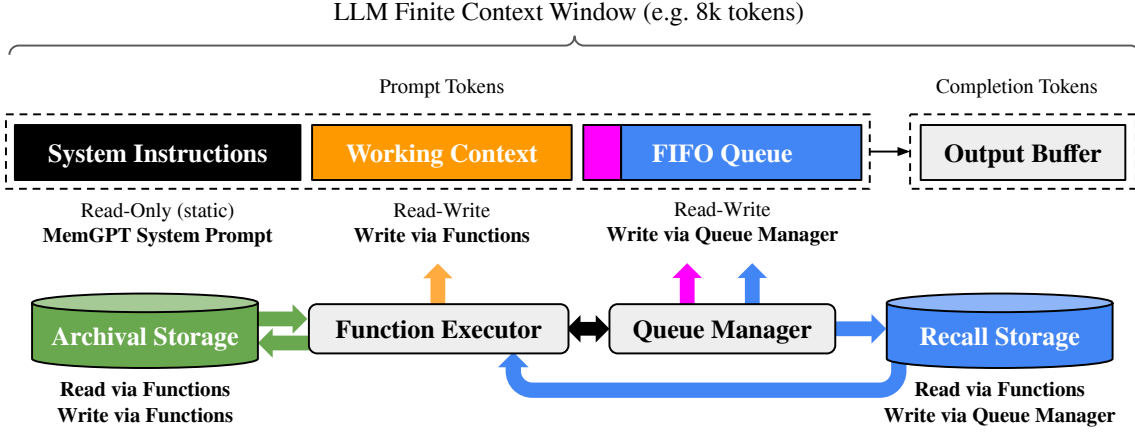


Figure 3. In MemGPT, a fixed-context LLM processor is augmented with a hierarchical memory system and functions that let it manage its own memory. The LLM’s prompt tokens (inputs), or *main context*, consist of the system instructions, working context, and a FIFO queue. The LLM completion tokens (outputs) are interpreted as function calls by the function executor. MemGPT uses functions to move data between main context and *external context* (the archival and recall storage databases). The LLM can request immediate follow-up LLM inference to chain function calls together by generating a special keyword argument (`request_heartbeat=true`) in its output; function chaining is what allows MemGPT to perform multi-step retrieval to answer user queries.

the queue to reinsert them into the LLM’s context window.

The queue manager is also responsible for controlling context overflow via a queue eviction policy. When the prompt tokens exceed the ‘warning token count’ of the underlying LLM’s context window (e.g. 70% of the context window), the queue manager inserts a system message into the queue warning the LLM of an impending queue eviction (a ‘memory pressure’ warning) to allow the LLM to use MemGPT functions to store important information contained in the FIFO queue to working context or **archival storage** (a read/write database storing arbitrary length text objects). When the prompt tokens exceed the ‘flush token count’ (e.g. 100% of the context window), the queue manager flushes the queue to free up space in the context window: the queue manager evicts a specific count of messages (e.g. 50% of the context window), generates a new recursive summary using the existing recursive summary and evicted messages. Once the queue is flushed, the evicted messages are no longer in-context and immediately viewable to the LLM, however they are stored indefinitely in recall storage and readable via MemGPT function calls.

2.3. Function executor (handling of completion tokens)

MemGPT orchestrates data movement between main context and external context via function calls that are generated by the LLM processor. Memory edits and retrieval are entirely self-directed: MemGPT autonomously updates and searches through its own memory based on the current context. For instance, it can decide when to move items between contexts (e.g. when the conversation his-

tory is becoming too long, as show in Figure 1) and modify its main context to better reflect its evolving understanding of its current objectives and responsibilities (as shown in Figure 3). We implement self-directed editing and retrieval by providing explicit instructions within the system instructions that guide the LLM on how to interact with the MemGPT memory systems. These instructions comprise two main components: (1) a detailed description of the memory hierarchy and their respective utilities, and (2) a function schema (complete with their natural language descriptions) that the system can call to access or modify its memory.

During each inference cycle, LLM processor takes main context (concatenated into a single string) as input, and generates an output string. This output string is parsed by MemGPT to ensure correctness, and if the parser validates the function arguments the function is executed. The results, including any runtime errors that occur (e.g. trying to add to main context when it is already at maximum capacity), are then fed back to the processor by MemGPT. This feedback loop enables the system to learn from its actions and adjust its behavior accordingly. Awareness of context limits is a key aspect in making the self-editing mechanism work effectively, to this end MemGPT prompts the processor with warnings regarding token limitations to guide its memory management decisions. Additionally, our memory retrieval mechanisms are designed to be cognizant of these token constraints and implement pagination to prevent retrieval calls from overflowing the context window.